Multi-Agent Coordination Patterns and MCP

Nov 6, 2026 · @Ashutosh

A multi-agent system splits work across specialised agents and needs a coordination pattern to decide who acts next and how results flow. Four patterns cover most designs: supervisor, hierarchical, network and sequential. MCP (Model Context Protocol) is not one of these patterns: it is the open standard that connects each agent to its tools and data, while its companion protocol A2A connects agents to each other.

1. When to use multiple agents

Use several agents only when one agent with good tools has clearly hit its limits; every extra agent adds model calls, latency and new ways to fail.

Signs that splitting helps:

- **Too many tools**: one agent with dozens of tools picks the wrong one more often. Giving each agent a small, focused toolset improves accuracy.
- **Distinct expertise**: tasks need different instructions or models, such as research, coding and review.
- **Context overload**: a sub-task would flood the main context with detail. A sub-agent can do the work in its own context and return only a summary.
- **Parallel work**: independent sub-tasks can run at the same time.
- **Organisational boundaries**: agents owned by different teams or vendors must cooperate.

Signs to stay with one agent: the task is short, the steps are tightly coupled, or every agent would need the same shared context anyway.

3. Comparing the patterns

The choice is a trade-off between central control and flexibility: the more freedom agents have to talk to each other, the harder the system is to predict and debug.

Pattern	Who decides the next step	Strengths	Weaknesses	Best fit
Supervisor	One central agent	Easy to follow and debug; clear ownership	Supervisor is a bottleneck and an extra model call per step	Most business tasks with 3 to 8 specialists
Hierarchical	Supervisors at each level	Scales to many agents; teams stay small	More layers, more latency and cost	Large systems split by domain or team
Network (peer-to-peer)	Each agent picks who goes next	Flexible; no single bottleneck	Can loop or drift; hardest to trace	Open-ended collaboration, brainstorming
Handoff (swarm)	The active agent transfers control	Natural for conversations; one agent talks to the user at a time	Needs clear rules for when to hand off	Customer support routing, triage
Sequential (pipeline)	Fixed order set in code	Predictable, cheap, easy to test	No adaptation to the input	Repeatable processes such as draft, review, publish
Parallel (fan-out / fan-in)	Code splits and merges	Fast on independent sub-tasks	Needs a merge step; results can conflict	Research across many sources, multi-file analysis

In practice these combine: a supervisor whose specialists each run a fixed pipeline, or a pipeline with one parallel research stage. Shared failure modes to plan for are agents looping between each other, lost context at handoffs, and cost growing with every extra hop.

4. MCP and A2A

MCP and A2A are complementary protocols, not coordination patterns: MCP is the vertical link from an agent down to its tools and data, and A2A is the horizontal link between agents. Both are now governed by the Agentic AI Foundation (AAIF) under the Linux Foundation (A2A blog).

Model Context Protocol (MCP)

MCP is an open standard for connecting an AI application to external tools and data through one interface, often compared to a USB-C port for AI. Anthropic released it in November 2024 and donated it to the newly formed AAIF in December 2025 (Taskade wiki).

- An **MCP server** wraps a system (a database, GitHub, Google Drive, an internal API) and exposes three kinds of capability: **tools** the model can call, **resources** it can read, and **prompts** (reusable templates).
- An **MCP client** inside the agent's host application discovers what a server offers and calls it.
- Any MCP-compatible agent can use any MCP server, so a tool integration is written once instead of once per framework or model.

In a multi-agent system, MCP is how each specialist gets its toolset. A billing agent connects to the billing MCP server and a support agent to the ticketing server, which also enforces least privilege per agent.

Agent2Agent protocol (A2A)

A2A is an open standard for agents built on different frameworks or by different vendors to discover each other and delegate work. Google introduced it in April 2025 and donated it to the Linux Foundation in June 2025 (SD Times). The specification reached v1.0 in March 2026 and A2A joined the AAIF in August 2026. Each agent publishes an **agent card** describing its skills and how to reach it; other agents read the card and send it **tasks**. LangGraph, CrewAI, Pydantic AI and other frameworks support it (A2A blog).

How they fit together

	MCP	A2A
Connects	Agent → tools and data	Agent ↔ agent
Direction	Vertical	Horizontal
Unit of work	A tool call or a resource read	A task delegated to another agent
Discovery	Server lists its tools, resources, prompts	Agent card lists skills and endpoint

	MCP	A2A
Origin	Anthropic, November 2024	Google, April 2025
Governance today	AAIF, Linux Foundation	AAIF, Linux Foundation

Inside one application, the coordination patterns above are usually implemented directly in a framework such as LangGraph, with MCP supplying each agent's tools. A2A becomes relevant when the agents cross a boundary, such as a company's agent delegating to a supplier's agent built on a different stack. A simpler middle ground is to expose a sub-agent as an MCP tool, so the calling agent treats it like any other tool.

5. Implementing the patterns in LangGraph

In LangGraph each agent is a node or a sub-graph, and the pattern is just the shape of the edges between them. The two most common building blocks:

Sub-agents as tools (supervisor). Wrap each specialist as a tool and give those tools to a coordinating agent. The supervisor calls a specialist, gets back only its final result, and decides what to do next. This keeps each specialist's working context out of the supervisor's.

```
from langchain.agents import create_agent
from langchain.tools import tool

researcher = create_agent("anthropic:claude-sonnet-4-5", tools=[web_search])
writer = create_agent("anthropic:claude-sonnet-4-5", tools=[])

@tool
def research(question: str) -> str:
    """Research a question and return a short summary of findings."""
    out = researcher.invoke({"messages": [("user", question)]})
    return out["messages"][-1].content

@tool
def write(brief: str) -> str:
    """Write a polished draft from a brief."""
    out = writer.invoke({"messages": [("user", brief)]})
    return out["messages"][-1].content
```

```
supervisor = create_agent(
    "anthropic:claude-sonnet-4-5",
    tools=[research, write],
    system_prompt="Plan the task, delegate research and writing, then check
the result.",
)
```

Handoffs (network or swarm). Each agent is a node in one graph. A node returns a `Command(goto="billing_agent", update={...})` to pass control and state to another agent, so the active agent decides who goes next. LangChain also publishes ready-made `langgraph-supervisor` and `langgraph-swarm` packages for these two shapes.

Other patterns map directly onto graph features: a **pipeline** is a chain of fixed edges, **parallel** fan-out uses several edges from one node (or the `Send` API for a variable number of branches) that join at a merge node, and a **hierarchy** nests a supervisor graph inside another as a sub-graph. Specialists can load their tools from MCP servers through LangChain's MCP adapters, and a LangGraph agent can be exposed to outside systems over A2A.

Code is illustrative; check the current LangChain docs for exact imports and model names.

Sources

- A New Chapter for A2A: Joining the Agentic AI Foundation, A2A Protocol blog, 27 August 2026
- Google's Agent2Agent protocol finds new home at the Linux Foundation, SD Times
- Model Context Protocol (MCP), Taskade wiki
- LangChain and LangGraph Agent Frameworks Reach v1.0 Milestones, LangChain blog, 22 October 2025
- LangChain documentation